

稀疏向量推荐系统：模型、算法与系统级优化

王锦政 2024270223

2026.4

问题建模与稀疏表示 (Problem Formulation)

背景与动机

在工业级推荐系统中，用户兴趣向量通常具有极高的维度 d ，但非零项 s 极少 ($s \ll d$)。传统稠密向量点积会产生大量无效的“零乘法”。

稀疏存储设计

- **数学表示:** 用户 u 表示为有序稀疏列表 $V_u = \{(idx_1, val_1), \dots, (idx_s, val_s)\}$ 。
- **约束:** 索引按升序排列 $idx_1 < idx_2 < \dots < idx_s$ 。
- **复杂度收益:** 空间复杂度由 $\mathcal{O}(d)$ 降至 $\mathcal{O}(s)$ 。

核心算法：稀疏余弦相似度 (Sparse Cosine Similarity)

利用有序特性，采用双指针法 (Two-Pointer Approach) 计算精确余弦相似度，避免 $\mathcal{O}(d)$ 的遍历。

Algorithm 1 SparseCosine(A, B)

```
1:  $i \leftarrow 0, j \leftarrow 0, dot \leftarrow 0$ 
2: while  $i < |A|$  and  $j < |B|$  do
3:   if  $A[i].idx == B[j].idx$  then
4:      $dot \leftarrow dot + A[i].val \times B[j].val$ 
5:      $i \leftarrow i + 1, j \leftarrow j + 1$ 
6:   else if  $A[i].idx < B[j].idx$  then
7:      $i \leftarrow i + 1$ 
8:   else
9:      $j \leftarrow j + 1$ 
10:  end if
11: end while
12: return  $dot / (norm(A) \times norm(B))$ 
```

Top- k 检索策略优化

给定候选集大小 n ，目标用户需找出相似度最高的前 k 个物品/用户。

基准算法 (Baseline: Full Sort)

- 流程: 全量计算 $\rightarrow$ 全量降序排序 $\rightarrow$ 截断前 k 。
- 复杂度: $\mathcal{O}(ns + n \log n + k)$ 。

优化算法 (Optimized: Min-Heap)

- 流程: 维护容量为 k 的最小堆，在线性扫描相似度时动态更新。
- 复杂度: $\mathcal{O}(ns + n \log k + k)$ 。
- 理论优势: 由于 $k \ll n$ ，排序开销被极大压缩。

实验复现 I: Min-Heap 优化效果分析

实验设置: $n = 30000, d = 10000, k = 10$ 。测试不同稀疏度 s 下的平均延迟 (ms)。

s (非零项)	Baseline (ms)	Min-Heap (ms)	Speedup
5	1.055	0.793	1.331x
10	2.025	1.604	1.262x
40	9.145	8.873	1.031x
160	39.319	37.033	1.062x
320	75.826	75.402	1.006x

结论:

- 当 s 较小 ($s \leq 10$) 时, Min-Heap 加速比显著 (1.33x)。
- 随着 s 增大, 相似度计算 $\mathcal{O}(ns)$ 成为绝对性能瓶颈, 堆排序的收益被边际化。

算法层进阶优化：打破 $\mathcal{O}(n)$ 瓶颈

为了消除 $\mathcal{O}(ns)$ 中的全库扫描开销，引入倒排索引与两阶段检索。

1. 倒排精确检索 (Inverted Exact Search)

- 机制：构建 Feature $\rightarrow$ User List 索引。查询时仅访问目标用户非零特征命中的候选人。
- 收益：平均加速约 53.8x。

2. 两阶段检索 (Two-Stage Recall & Rerank)

- 机制：阶段一利用受限倒排快速召回候选集；阶段二使用精确余弦做精排 Top- k 。
- 权衡：提速约 13.5x，在 $s \leq 160$ 时保持近乎 100% 的 Recall@10。

代码与系统层优化 (Systems-Level Optimization)

在底层实现上，结合计算机体系结构特性进行常数级极致优化：

1. **内存布局 (Memory Layout)**: 从 AOS (每用户独立 Vector) 转换为 CSR (Compressed Sparse Row) 格式，大幅提升空间局部性与 Cache 命中率。
2. **数值计算 (Numerical Types)**: 将双精度 'double' 降维至单精度 'float'，降低内存带宽压力并提升 SIMD 吞吐潜力。
3. **预计算 (Precomputation)**: 离线缓存 $inv_norm = 1/\|u\|$ ，将查询时的除法转化为代价更低的乘法链。

注：经验证，上述优化在 $Top-k$ ($Overlap@10$) 严格达到 1.000，即无质量损失。

实验复现 II：算法与系统级优化叠加 (Ablation & Combined)

实验设置: $n = 20000, d = 10000, k = 10$ 。评估代码级与算法级叠加的端到端延迟。

s	Baseline	Code-Only (CSR+Float)	Algo-Only (Inverted)	Combined
20	2.818 ms	2.295 ms	0.036 ms	0.029 ms
40	5.933 ms	4.899 ms	0.120 ms	0.099 ms
80	12.242 ms	10.103 ms	0.488 ms	0.424 ms
160	24.303 ms	19.970 ms	1.119 ms	0.950 ms
320	48.754 ms	40.119 ms	1.729 ms	1.388 ms

性能分析: 代码层优化稳定提供约 1.2x 常数级加速；结合算法层优化后，总体获得最高 49.19x 的端到端性能飞跃。两者在正交维度上实现了完美叠加。

结论与工程指导意义 (Conclusion & Takeaways)

针对大规模稀疏推荐系统，基于本文复现与优化可得出以下实践准则：

- **小 s 、小 k 场景：**

采用 Top- k 最小堆 + 双指针稀疏余弦，即可获得极佳性价比。

- **大规模候选集 (n 极大) 场景：**

必须引入倒排索引或两阶段召回，打破全扫描的 $\mathcal{O}(n)$ 计算壁垒。

- **底层系统架构：**

CSR 连续内存布局、Float 单精度以及代数预计算应作为**系统基建**，其提供的 $\sim 20\%$ 加速在工程上完全免费且与任何上层算法正交。